

POEChatBotApplication README

Application Overview

The POEChatBotApplication is a Windows Presentation Foundation (WPF) desktop application that combines task management functionality with an interactive chatbot interface. The application features a cyberpunk-inspired user interface with modern styling elements including gradient backgrounds, glowing effects, and hexagonal button controls.

Technical Specifications

- Development Framework: .NET 6.0
- UI Framework: Windows Presentation Foundation (WPF)
- Architecture: MVVM (Model-View-ViewModel) pattern
- Dependencies:
 - .NET Core Runtime 6.0+
 - Windows 10/11 operating system

Key Features

1. Task Management System

- Create, read, update, and delete (CRUD) operations for tasks
- Task attributes:
 - Title (required field)
 - Description (optional)
 - Due date with date picker control
- **Visual task cards with:**
 - Title in accent color
 - Description in secondary text
 - Due date with icon indicator
- Validation for required fields

2. User Interface Components

- Cyberpunk-themed controls:
 - Hexagonal action buttons
 - Gradient-filled headers
 - Glowing border effects
 - Circuit board-inspired design elements
- Responsive layout with:
 - Task creation panel (left)
 - Task display list (right)
 - Visual divider

3. Technical Implementation Details

- Data Binding:
 - ObservableCollection for dynamic task updates
 - Two-way binding for edit functionality
- Controls:
 - ListBox for task selection
 - DatePicker for due date selection
 - Custom styled TextBox controls
- Converters:
 - ToUpperConverter for text formatting
 - Date formatting converters

Installation Requirements

System Requirements:

- Operating System: Windows 10 or later
- .NET Runtime: Version 6.0 or higher
- Recommended Hardware:
 - 4GB RAM minimum
 - 500MB available disk space

Development Environment Setup:

1. Install Visual Studio 2022 with:
 - .NET desktop development workload
 - C# programming language support
2. Clone repository:
git clone <https://repository.url/POEChatBotApplication.git>
3. Restore NuGet packages
4. Build solution

Application Structure

The solution contains the following key components:

Views/

- TaskView.xaml - Main task management interface
- TaskView.xaml.cs - Code-behind for task operations
- QuizView.xaml - Chatbot interaction interface
- QuizView.xaml.cs - Chatbot implementation code

Models/

- Task.cs - Data model for task items
 - Properties: Title, Description, ReminderDate
 - Data validation attributes

Converters/

- ToUpperConverter.cs - Value converter for text formatting

Resources/

- Styles.xaml - Contains all UI styling resources
- Brushes.xaml - Color and gradient definitions

Usage Instructions

Task Management:

1. Creating a Task:
 - Enter task title in the Title field
 - Optionally add a description
 - Select a due date using the date picker
 - Click "DEPLOY TASK" button
2. Editing a Task:
 - Select task from the list
 - Click the edit (✎) button
 - Modify fields as needed
 - Changes are saved automatically
3. Deleting a Task:
 - Select task from the list
 - Click the delete (X) button
 - Confirm deletion in the dialog

Troubleshooting

Common Issues and Solutions:

1. UI Rendering Problems:
 - Symptom: Controls not displaying properly
 - Solution: Verify .NET runtime version matches build target
2. Data Binding Errors:
 - Symptom: Task list not updating
 - Solution: Ensure ObservableCollection is used for Tasks property
3. Converter Not Found:
 - Symptom: "ToUpperConverter could not be resolved" error

- Solution: Verify namespace declaration in XAML:
xmlns:converters="clr-namespace:POEChatBotApplication.Converters"
- Solution: Verify resource declaration:
<converters:ToUpperConverter x:Key="ToUpperConverter"/>

4. Selection Issues:

- Symptom: SelectedItem not working
- Solution: Confirm ListBox is used instead of ItemsControl

Development Notes

1. Styling Implementation:

- All visual styles are defined in resource dictionaries
- Custom control templates for buttons and inputs
- VisualStateManager used for interactive states

2. Data Flow:

- UI → ViewModel → Model binding
- INotifyPropertyChanged implementation
- IValueConverter interfaces for data transformation

3. Best Practices:

- Separation of concerns between UI and logic
- Resource dictionaries for maintainable styling
- ObservableCollection for dynamic data

License Information

This application is licensed under the MIT License.